

Informe Técnico Red Team — Ronda 1

Auditoría de Seguridad de la API · Inyección NoSQL

Proyecto	Operación Centinela	De	Equipo Red Team · Ciberseguridad
Cliente	NovaPay (ficticio)	Para	Equipo Data Science · Blue Team
Fecha	25 de mayo de 2026	Clasificación	Confidencial · Uso interno

Resumen ejecutivo

Se identificó una vulnerabilidad **crítica de inyección NoSQL (Array Injection)** en el endpoint `/trans/` de la API de NovaPay. El vector permite un **bypass total de la autorización**: cualquier usuario autenticado puede recuperar las transacciones de toda la base de datos, no solo las propias. El impacto confirmado es de divulgación de información (information disclosure); no se probó la escritura. Se documenta como hallazgo de auditoría con sus recomendaciones de mitigación y **no se explotó como vector adversarial**, dada la ausencia de copias de seguridad de la base de datos y la proximidad de la fecha de entrega.

Críticas	Altas	Medias	Endpoints auditados
1	1	2	6

1. Alcance

La auditoría cubrió el frente de inyección NoSQL sobre la API de NovaPay, con foco en la capa de persistencia basada en Valkey (fork de Redis). El objetivo fue identificar vectores de inyección en las consultas de filtrado que pudieran comprometer la confidencialidad o la integridad de las transacciones.

Vectores probados

Técnica	Carga de prueba
Wildcard injection	<code>tipo_transaccion=*, =?, =T*</code>
Command injection	secuencias <code>KEYS *</code> / <code>SCAN 0</code> (URL-encoded)
Array injection	<code>tipo_transaccion[]=VALOR</code> (sintaxis PHP/Node.js)
Object injection	<code>tipo_transaccion[key]=value</code>
Nested arrays	<code>tipo_transaccion[][]=test</code>
Operadores NoSQL	<code>\$ne</code> , <code>\$gt</code> en query string

Endpoints auditados

Método	Endpoint	Descripción
GET	<code>/trans/?tipo_transaccion=...</code>	Filtro principal · vector primario

GET	/trans/?analista=...	Parámetro secundario
GET	/trans/?revisado=...	Parámetro secundario
GET	/trans/usuarios/{id}/trans	Endpoint de usuario específico
PATCH	/trans/update/{id}	Update con operadores
POST	/predict/	Endpoint de predicción

2. Resultados

Entrada / vector	Respuesta	Observación
tipo_transaccion=TRANSFER	HTTP 500	Error interno · fallo de pattern-matching
tipo_transaccion=*	HTTP 500	Wildcard sin escape provoca error
tipo_transaccion[]=CUALQUIER_VALOR	HTTP 200 · 500 registros	BYPASS TOTAL
/trans/usuarios/{id}/trans (legítimo)	HTTP 200 · 3 registros	Endpoint correcto · aislado
Wildcards largos	Timeout 7,52 s	Procesamiento de patrones en Redis

3. Hallazgos

Hallazgo 1 — Array Injection en tipo_transaccion · **CRÍTICA**

Bypass de autorización con divulgación de información. Una petición con el parámetro de filtro enviado como array devuelve la totalidad de los registros, ignorando el aislamiento por usuario.

```
GET /trans/?tipo_transaccion[]=NOEXISTE&limite=500 HTTP/1.1
Host: api.novapay.internal
Authorization: Bearer [token_usuario_normal]

# Respuesta: HTTP 200 OK
# Cuerpo: 500 registros de transacciones de TODOS los usuarios
```

Endpoint	Registros	Ámbito
/trans/usuarios/57c265cc-.../trans	3	Solo el usuario autenticado ✓
/trans/?tipo_transaccion[]=X	500	Toda la base de datos ✗ (bypass)

Causa raíz: confusión de tipos (*type confusion*). El backend espera un string, pero al recibir un array la lógica de filtrado falla de forma silenciosa y retorna todos los registros. La validación de tipos ocurre después de construir la consulta a Valkey, o el driver maneja el array de forma inesperada saltándose por completo la cláusula de filtrado.

Hallazgo 2 — Múltiples parámetros vulnerables · ALTA

Las pruebas con `es_fraude[]=true` y `revisado[]=test` también retornaron registros hasta el límite solicitado, lo que indica que el vector de array injection no se limita a `tipo_transaccion`: la superficie de ataque es más amplia.

Hallazgo 3 — Denegación de servicio por wildcards · MEDIA

Los wildcards de longitud elevada provocan un timeout de 7,52 s, lo que evidencia operaciones `SCAN/MATCH` en Valkey sin límite de tiempo adecuado. Un atacante podría lanzar múltiples peticiones concurrentes para degradar el servicio.

Hallazgo 4 — Gestión verbosa de errores · MEDIA

Los errores HTTP 500 ante entradas como `tipo_transaccion=TRANSFER` (valor esperado válido) revelan comportamiento interno de la base de datos, confirman el uso de lógica `SCAN/MATCH` de Redis y facilitan el fingerprinting de la arquitectura interna.

4. Cuadro consolidado

Severidad	Vulnerabilidad	Tipo	Evidencia
CRÍTICA	Authorization Bypass	Bug	<code>tipo_transaccion[]=X</code> → 500 registros de todos los usuarios
ALTA	Information Disclosure	Bug	500 vs 3 registros: datos cross-user accesibles
MEDIA	DoS potencial	Debilidad	Wildcards largos → timeout 7,52 s
MEDIA	Gestión verbosa de errores	Debilidad	HTTP 500 revela la arquitectura interna

Nota importante

El modelo XGBoost no es vulnerable en sí mismo. La vulnerabilidad reside en la capa de acceso a datos que alimenta al modelo. Es un bug explotable, no una limitación estructural del pipeline de ML.

5. Remediación

R1 · Sanitización estricta de tipos

Validar que `tipo_transaccion` sea un string antes de construir la consulta a Valkey.

```
if not isinstance(tipo_transaccion, str):
    raise ValidationError('tipo_transaccion must be string')
```

R2 · Whitelist de valores permitidos

Dado que el parámetro tiene un conjunto cerrado de valores, aplicar una lista blanca explícita.

```
ALLOWED_TYPES = {'TRANSFER', 'PAYMENT', 'WITHDRAWAL'}
if tipo_transaccion not in ALLOWED_TYPES:
    return []
```

R3 · Escapado de caracteres especiales de Redis

Si se requiere pattern-matching, escapar los wildcards de Redis.

```
tipo = tipo.replace('*', r'\*').replace('?', r'\?')
```

R4 · Separación de responsabilidades: filtrado vs. predicción

Asegurar que el modelo no entrene con datos obtenidos mediante consultas vulnerables (posible data leakage si el atacante manipula el conjunto de entrenamiento).

R5 · Rate limiting específico en `/trans/`

Prevenir la extracción masiva por bypass iterativo (el atacante puede iterar con `limite=500`).

6. Limitaciones y conclusión

- No se confirmó inyección de comandos Redis arbitrarios; los intentos con secuencias de control fueron rechazados o no produjeron evidencia clara de ejecución.
- No se probó la escritura en la base de datos: el impacto confirmado se limita a divulgación de información.
- No se auditó en profundidad `/auth/crear-analista-inicial` (solo identificado en el OpenAPI).
- Las pruebas fueron manuales y dirigidas, sin fuzzing automatizado exhaustivo; no se verificó si el vector afecta a otros endpoints con parámetros similares (p. ej. `/clientes/`).

Se identificó una vulnerabilidad crítica de NoSQL Array Injection que permite el bypass total de autorización y la extracción masiva de datos transaccionales. Requiere corrección antes de Ronda 2. El hallazgo se entrega al Blue Team para su mitigación; el Red Team optó deliberadamente por no explotarlo como vector de evasión.